

Programação de Sistemas Linux

Aula 06 — Inicialização do Sistema Linux (Boot)

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Sumário

- 1 Abertura
- 2 BIOS/UEFI e o bootloader
- 3 O kernel assume o controle
- 4 systemd: units, targets, dependências
- 5 Runlevels e journald
- 6 Encerramento



Notes

Notes

Abertura

Notes

Onde estamos

Começa a Unidade 2: Programação Linux

As Aulas 1–5 construíram a base (arquitetura, ferramentas, syscalls). A partir de hoje, o foco vira **programar** o sistema Linux de verdade.

Aula 5 (feito)

- Mecanismo de **syscall**
- Syscall vs. função de biblioteca
- `errno` e tratamento de erros

Aula 6 (hoje)

- Como o Linux **liga**:
BIOS/UEFI → GRUB → kernel
→ `systemd`
- Criar e gerenciar serviços com `systemd`
- Ler logs de boot com `journalctl`

Notes



Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Descrever as **quatro fases** do boot de um sistema Linux
- ✓ Editar parâmetros de **kernel** via GRUB, temporária e permanentemente
- ✓ Criar, habilitar e gerenciar um **serviço systemd** simples
- ✓ Investigar o boot e os serviços com `journalctl`

Regra de hoje

Todo processo que roda no seu Linux — inclusive o seu programa em C depois de compilado — é, no fim das contas, filho do processo que a gente vai estudar hoje: o `systemd` (PID 1).



Notes

BIOS/UEFI e o bootloader

Notes

Do botão ligado ao login: quatro fases



- Cada fase **entrega o controle** para a próxima — se uma falha, as seguintes nem começam
- Hoje vamos abrir cada uma dessas caixas, uma de cada vez



Notes

BIOS vs. UEFI

	BIOS (legado)	UEFI (atual)
Tabela de partição	MBR	GPT (discos >2TB)
Modo de execução	16 bits	32/64 bits
Interface	texto	pode ter menu gráfico
Segurança	nenhuma nativa	Secure Boot

- A firmware é o **primeiro código** que roda ao ligar a máquina — antes de qualquer sistema operacional existir
- Ela testa o hardware básico (POST) e entrega o controle ao **bootloader** gravado no disco



Notes

GRUB2: editando o boot na hora

No menu do GRUB, pressione e sobre uma entrada para editar os parâmetros **só daquele boot** (não altera nada permanentemente):

```
1 linux /boot/vmlinuz-... root=UUID=... ro quiet splash
2                                     ^^^^^^ ^^^^^^
3                                     adicione/remova parametros aqui
4
```

- Remover `quiet splash` mostra o **log de boot completo** — ótimo para depurar
- `Ctrl+X` ou `F10` inicia o boot com os parâmetros editados
- Desligou/reiniciou? A mudança **some** — é só para aquela vez



Notes

GRUB2: tornando a mudança permanente

```
1 $ sudo nano /etc/default/grub
2   # GRUB_CMDLINE_LINUX_DEFAULT="quiet splash meu_parametro=1"
3
4 $ sudo update-grub                # Debian/Ubuntu
5 $ sudo grub2-mkconfig -o /boot/grub2/grub.cfg # Fedora/RHEL
6
```

Faça isso em uma VM (com *snapshot*)

Editar a configuração de boot é uma das poucas mudanças que podem deixar a máquina sem iniciar se algo sair errado. Pratique em uma VM e tire um *snapshot* antes.



Notes

O kernel assume o controle

Notes

O que o kernel faz nos primeiros instantes

- 1 O GRUB carrega o kernel **comprimido** (`vmlinuz`) na memória
- 2 O kernel se **descomprime** e inicializa drivers essenciais
- 3 Monta um sistema de arquivos **temporário** na RAM: o `initramfs` — tem só o necessário para achar o disco real
- 4 Monta o sistema de arquivos **raiz** de verdade (o seu `/`)
- 5 Executa o primeiro processo em espaço de usuário: `/sbin/init` (hoje, quase sempre `systemd`) — vira o **PID 1**

Por que existe o `initramfs`?

O kernel pode precisar de um driver (ex.: para um disco NVMe ou um volume criptografado) só para **conseguir montar** o disco raiz. O `initramfs` carrega esse driver antes de precisar dele.

Notes



Quem é responsável por quê

Fase	Responsabilidade
Firmware (BIOS/UEFI)	testar hardware básico, achar o bootloader
Bootloader (GRUB2)	carregar o kernel e o <code>initramfs</code> na memória
Kernel	inicializar drivers, montar o <code>/ real</code> , lançar o PID 1
Init (<code>systemd</code>)	subir todos os serviços do sistema, em ordem



Notes

systemd: units, targets, dependências

Notes

Definição

systemd é o processo de **init** usado pela maioria das distribuições Linux atuais. Ele sobe todo o resto do sistema como um conjunto de **unidades** (*units*) – descrições declarativas do que precisa rodar e de que outras unidades dependem.

Tipo de unit	Representa
.service	um processo/daemon (ex.: <code>ssh.service</code>)
.target	um agrupamento de units (ex.: <code>multi-user.target</code>)
.socket, .mount, .timer	socket, ponto de montagem, agendador



Notes

Explorando as units existentes

```

1 $ systemctl status           # visao geral do sistema
2 $ systemctl list-units --type=service # servicos ativos agora
3 $ systemctl list-units --type=target  # "grupos" de units
4 $ systemctl status ssh.service      # status de UM servico
5

```

- status mostra se está active (running), failed, inactive...
- Praticamente **tudo** que roda em segundo plano hoje é uma unit – rede, SSH, cron, até o servidor gráfico



Notes

Anatomia de um .service

```
1 [Unit]
2 Description=Monitor de espaço em disco (PSL221A09)
3 After=network.target
4
5 [Service]
6 Type=oneshot
7 ExecStart=/usr/local/bin/monitor-disco.sh
8
9 [Install]
10 WantedBy=multi-user.target
11
```

- [Unit]: metadados e **quando** essa unit deve subir em relação a outras
- [Service]: **o que** rodar e **como** (uma vez? sempre reiniciar?)
- [Install]: em qual *target* essa unit entra quando habilitada



Notes

Habilitando e rodando seu próprio serviço

```
1 $ sudo cp monitor-disco.service /etc/systemd/system/
2 $ sudo systemctl daemon-reload      # recarrega a lista de units
3 $ sudo systemctl start monitor-disco.service
4 $ systemctl status monitor-disco.service
5 $ sudo systemctl enable monitor-disco.service  # sobe automatico no
6      boot
```

- daemon-reload é obrigatório sempre que você **cria ou edita** um .service
- start roda **agora**; enable faz rodar em **todo boot futuro** — são independentes



Notes

Ordem vs. necessidade

After=/Before= controlam **ordem** de inicialização. Requires= diz que uma unit **precisa** da outra (falhou a outra, essa também falha). Wants= é uma dependência **fraca** — tenta subir a outra, mas segue em frente mesmo se ela falhar.

After= sem Requires= é o padrão mais comum: “depois dessa, mas não morro se ela falhar”.



Notes

Runlevels e journald

Notes

Runlevels (legado) viraram *targets*

Runlevel (SysV)	Target (systemd)
0 – desligar	poweroff.target
1 – modo single-user	rescue.target
3 – multiusuário, sem GUI	multi-user.target
5 – multiusuário, com GUI	graphical.target
6 – reiniciar	reboot.target

```
1 $ systemctl get-default      # qual target sobe por padrao
2 $ sudo systemctl isolate rescue.target # troca de target agora
3
```



Notes

journald: todos os logs, um lugar só

```
1 $ journalctl -b          # log do boot atual
2 $ journalctl -b -1       # log do boot ANTERIOR
3 $ journalctl -u ssh.service # so as mensagens de um servico
4 $ journalctl --since "10 min ago" # filtra por tempo
5 $ journalctl -f          # acompanha em tempo real (como tail
6                          -f)
```

- Substitui (na maioria das distros) os arquivos de texto soltos em `/var/log/*`
- Binário, indexado e **estruturado** – por isso dá para filtrar por serviço, tempo, prioridade, sem `grep` manual



Notes

Encerramento

Notes

Síntese da aula

Notes

Conceito	Pergunta que responde
BIOS/UEFI + GRUB	Como o computador acha e carrega o kernel?
Kernel + <code>initramfs</code>	Como o kernel chega até o disco raiz de verdade?
<code>systemd (units/targets)</code>	O que sobe, em que ordem, e com quais dependências?
<code>journalctl</code>	Onde eu vejo o que aconteceu durante o boot?

Da próxima vez que o PC travar no boot, vocês vão saber exatamente onde procurar.



Atividade prática de hoje

No laboratório, vocês vão:

- 1 Investigar o boot atual e o anterior com `journalctl -b`
- 2 Criar e habilitar um serviço `systemd` simples, do zero
- 3 Editar (com segurança, em VM) um parâmetro de kernel via GRUB

Guia da atividade

Todas as instruções, código-fonte e critérios de avaliação estão no **guia de atividade prática** do Laboratório 05.



Notes

Próxima aula

Aula 7 – Shell Scripting

- *Shebang*, variáveis, expansões
- Redirecionamento e pipes
- Estruturas de controle: `if`, `case`, `for`, `while`
- Funções, e um primeiro contato com `grep`/`sed`/`awk`

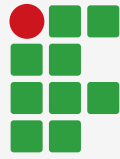
Bibliografia de apoio

- Manuais: `man bootup`, `man systemd`, `man systemd.service`, `man journalctl`
- Arch Wiki – `systemd` (<https://wiki.archlinux.org/title/Systemd>)
- GNU GRUB Manual – <https://www.gnu.org/software/grub/manual/grub/grub.html>



Notes

That's all folks!



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
Florianópolis



Notes

Notes
